QE Agentic AI Platform

Full User Manual

Version 0.1

Document History

Document Version

Date	Version	Changes	Author
16/02/2026	0.1	Initial draft version	Antima Jain

Review History

Date	Version	Reviewer
16/02/2026	0.1	

Table of Contents

1. Overview

The **QE Agentic Framework** is an intelligent automation platform designed to help Quality Engineering (QE) teams automatically interpret requirements in different formats and generate:

- **Test cases**
- **Test data**
- **Test scripts**

By combining **LLM-powered agents**, **RAG-driven knowledge retrieval**, and **workflow orchestration**, the tool accelerates QE processes and increases consistency, accuracy, and coverage.

---

2. System Architecture

The framework is hosted on cloud and can be currently accessed via
http://13.40.129.105:8501/

The architecture consists of the following major layers:

2.1 Web UI – Presentation Layer

- Provides the graphical interface to:
    - QE Agentic Framework
        - Input text requirements
        - Upload requirements in text file or MD file format
        - Generate Test cases
        - Download Test cases in JSON or CSV format
        - Generate Test scripts as a standalone solution (Upload your own test cases in excel format)
        - Generate Test scripts as an integrated solution (using test cases generated by the framework for the requirements you provided)
        - Generate Test Data in the form of JSON, CSV, SQL using either Test Cases or Test Scripts
    - QEAF LLM Configuration
        - LLM Provider
        - LLM Model
        - LLM API Key
        - Temperature
        - Max Tokens
- UI communicates with backend via API Gateway.

2.2 API Gateway (Fast API) – Service Layer

Responsible for:

- Validating requests from UI
- Routing to appropriate agent orchestrators
- Handling authentication where enabled

2.3 Agent Orchestration Layer

This is the core decision-making engine that coordinates QE agents:

Orchestrator Responsibilities

- Interpret user intent
- Delegate tasks to appropriate agents
- Consolidate outputs
- Maintain context between steps

Supported Agent Tasks

- requirement_interpreter
- task_case_generator
- task_data_generator
- task_script_generator

This enables sequential or parallel task execution depending on workload.

2.4 Agents Layer

Agents execute distinct QE automation tasks via task queues (RQ/Celery).

Available Agents

Agent	Function
Test Case Generator	Produces structured test cases based on requirements
Test Data Generator	Creates synthetic, boundary, or scenario-based test data
Test Script Generator	Produces automation-ready scripts (e.g., pytest, robot)
Task Queue Handler	Manages distributed workloads

2.5 LLM Interfaces

The tool integrates with multiple AI models:

- **OpenAI**
- **Google Gemini**

LLMs assist in:

- Natural language understanding
- Test design reasoning
- Script generation
- Data classification

3. Framework Setup

3.1 Introduction

This framework provides a **configurable, reusable platform** that enables teams to onboard **multiple projects and clients** with minimal effort. It is designed to be **deployment-agnostic**, supporting both **on-premises** and **cloud-hosted** environments, while allowing **client-specific customization** without changing the core framework.

The primary objective is to:

- Reduce project setup time
- Maintain a single core framework
- Enable client- and project-level flexibility
- Ensure consistency, scalability, and maintainability

3.2 Framework Architecture Overview

The framework follows a **core + configuration model**:

- **Core Framework**
  Contains common logic, UI components, workflows, validations, and reusable utilities. This layer remains unchanged across clients and projects.
- **Configuration Layer**
  Drives behaviour through configuration files or parameters such as:
  - Client details
  - Project metadata
  - Environment settings
  - Feature toggles
  - Tool integrations

This separation ensures **customization without code duplication**.

3.3 Supported Deployment Models

The framework supports the following deployment options:

3.3.1 On-Premises Deployment

- Hosted within the client's internal infrastructure
- Suitable for restricted or regulated environments
- Integrated with internal networks, repositories, and authentication systems

3.3.2 Cloud Deployment

- Deployed on cloud infrastructure (public or private)
- Supports scalability and remote access
- Suitable for distributed teams and multi-region usage

The same framework package can be used for both deployment models with **environment-specific configuration changes only**.

3.4 Project and Client Onboarding

3.4.1 Adding a New Client

To onboard a new client:

1. Create a **client-specific configuration** entry.
2. Define:
    - Client name and identifier
    - Supported environments (e.g., Dev, QA, UAT, Prod)
    - Access roles and permissions
3. Map the client to relevant projects.

No changes to the core framework are required.

3.4.2 Adding a New Project

For each project:

- Define project-level configuration such as:
    - Project name and ID
    - Associated client
    - Technology stack (if applicable)
    - Execution or workflow preferences

Each project inherits default framework behaviour, which can be overridden at the project level if required.

3.4.3 Configuration Management

Configuration is the **key customization mechanism** in the framework.

Typical configurable elements include:

- Environment URLs and endpoints
- Credentials and secrets (managed securely)
- Feature enablement/disablement
- Reporting preferences
- Integration settings (CI/CD, test management tools, etc.)

Best practices:

- Use **separate configuration files per environment**
- Avoid hard-coding client or project details
- Keep sensitive data externalized and secured

3.4.4 Environment Setup

The framework supports multiple environments per project.

Each environment configuration typically includes:

- Environment name (e.g., Dev, QA, Staging, Production)
- Deployment type (on-prem or cloud)
- Runtime parameters
- Logging and monitoring settings

This enables teams to:

- Use the same project setup across environments
- Promote configurations safely between environments

3.4.5 Client-Specific Customization

Client-specific requirements are handled through:

- Configuration overrides
- Optional feature flags
- Client-specific resource mapping

Examples:

- Enabling/disabling modules per client
- Custom naming conventions
- Client-specific workflows or validations

This approach ensures:

- Maximum reuse of the core framework
- Minimal risk during upgrades
- Easier maintenance and support

3.4.6 Security and Access Control

The framework supports role-based access and environment-level controls.

Key considerations:

- Separate access for different clients
- Restricted access to production environments
- Secure handling of credentials and secrets
- Auditability of user actions

Security configuration is aligned with the deployment environment (on-prem or cloud).

3.4.7 Maintenance and Upgrades

Because client and project customizations are configuration-driven:

- Core framework upgrades can be applied centrally
- Client-specific configurations remain unaffected
- Regression risk is minimized

Recommended approach:

- Validate upgrades in a lower environment
- Roll out changes progressively across clients

3.4.8 Troubleshooting and Support

Common troubleshooting steps:

- Validate configuration files
- Check environment-specific settings
- Review application logs
- Confirm access permissions

For client-specific issues, always verify that:

- Correct client and project configuration is selected
- Environment mappings are accurate

---

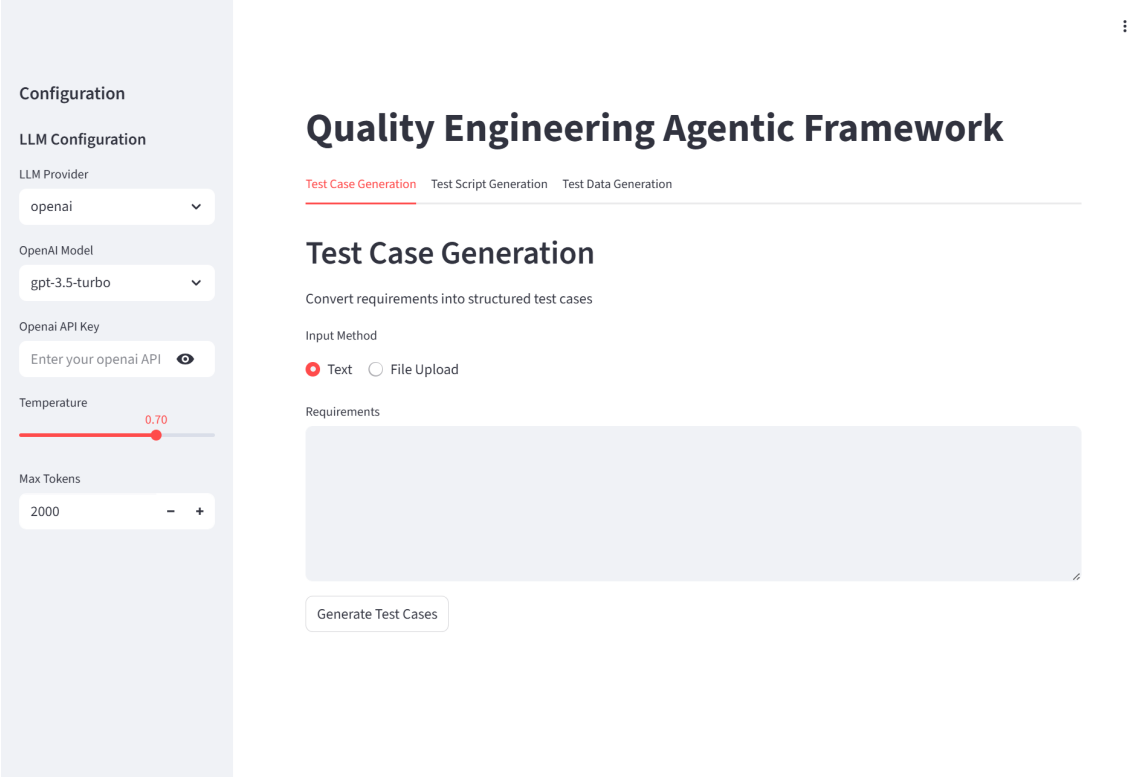
4. Using the QE Agentic Framework

This section walks you step-by-step through the interface hosted currently at **http://13.40.129.105:8501/**.

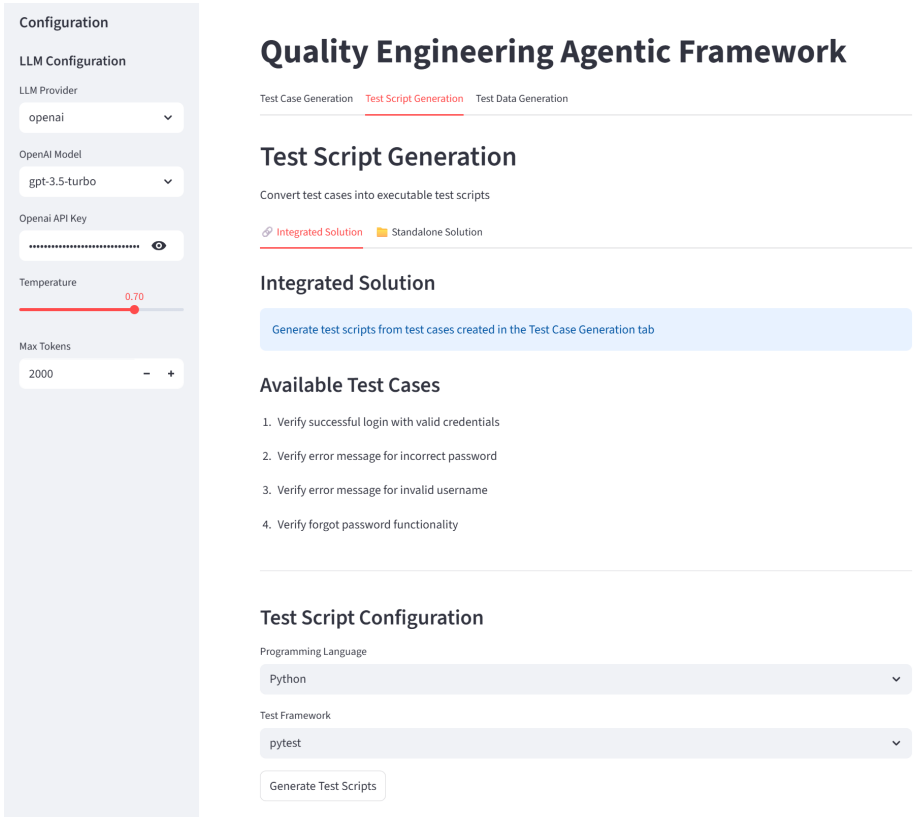
4.1 User Interface

When you open the tool, you will see:

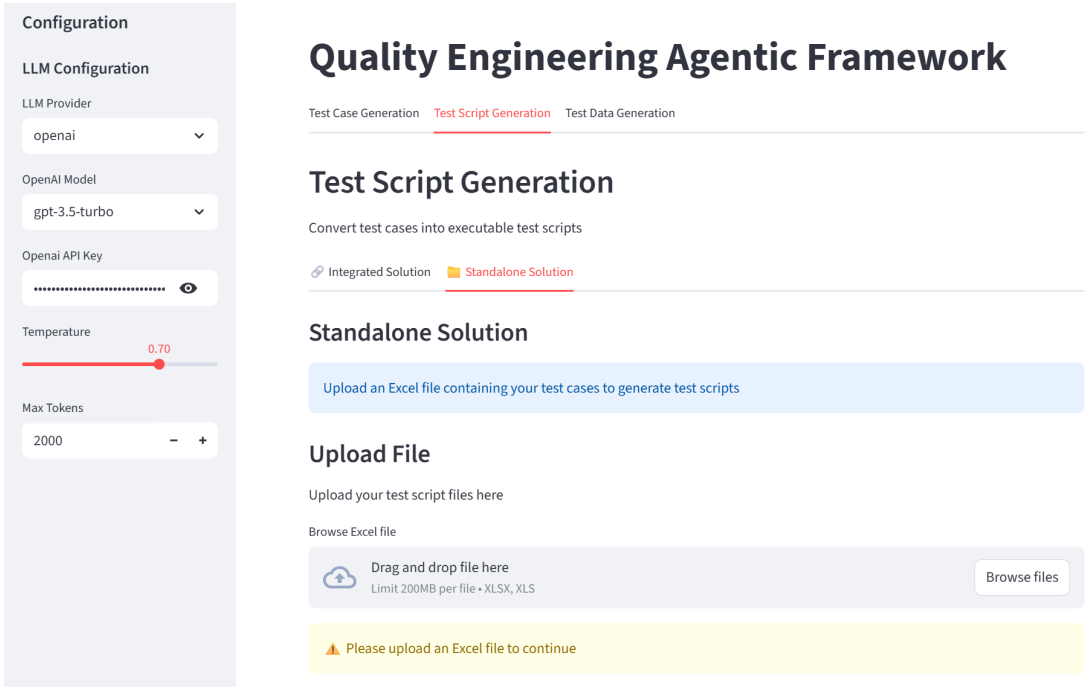
- Sidebar menu with available LLM configuration for the user to choose (as mentioned in section 2.1)
- Central workspace area with three tabs
    - **Test Case Generation** will have options for uploading input text or file upload



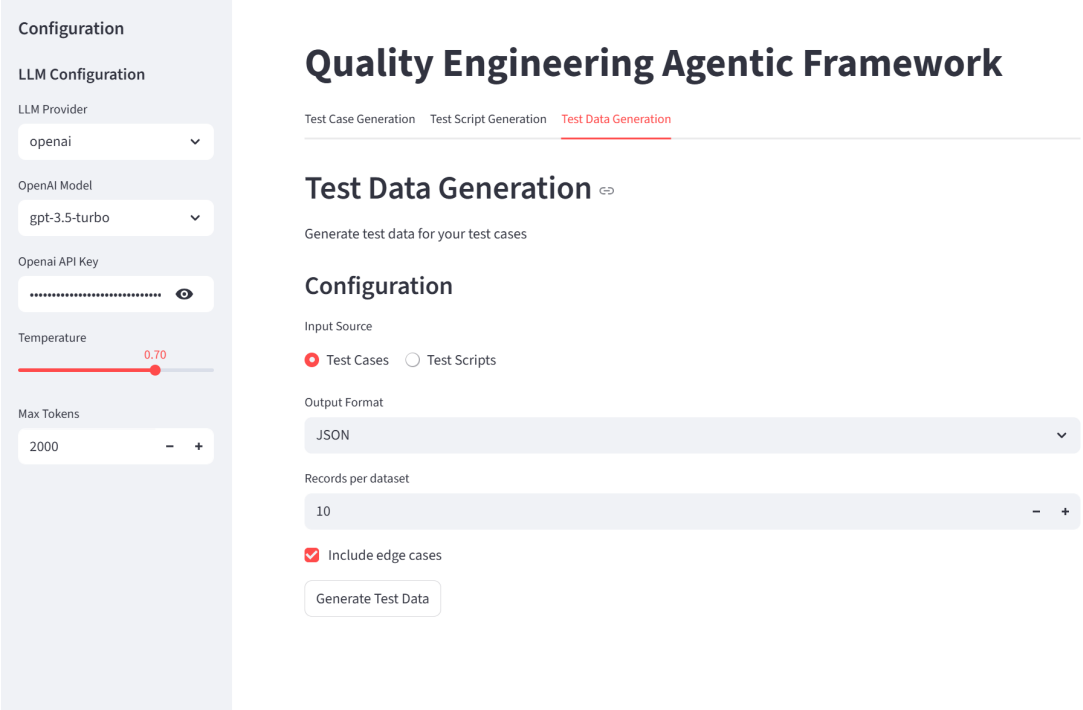
    - **Test Script Generation** will have two tabs each with options to generate Test Script with choice to choose the Programming language and the respective test framework.
        - Integrated Solution - using test cases generated by the framework for the requirements you provided



- Standalone solution - Upload your own test cases in excel format



- **Test Data Generation** to generate test data either via Test Cases or Test Scripts in different output format with option to include edge cases



4.2 Test Case Generation

Purpose

Automatically generate detailed, structured test cases.

Steps

1. Provide requirement(s) in text format or upload a .txt or .md
2. Select **"Generate Test Cases"**
3. ~~Choose test type (functional, boundary, negative, API, etc.)~~
4. Output includes:
    - ID
    - Title
    - Description
    - Preconditions
    - Actions
    - Expected Results

The generated test case could be downloaded as:

- CSV
- JSON
- ~~Test management systems (if integrated)~~

4.3 Test Script Generation

Purpose

Generate automation test scripts using your own or framework generated test cases for popular frameworks from different programming languages.

Supported Frameworks

- **Python**
  - Pytest
  - UnitTest
  - Robot Framework
- **JavaScript**
  - Jest
  - Mocha
  - Cypress
- **Java**
  - Junit
  - TestNG
  - Cucumber
- **C#**
  - NUnit
  - XUnit
  - MSTest

Steps

1. Provide test case input (either your own test cases into Standalone Solution tab or test cases generated by framework using the input requirements under Integrated Solution tab)
2. Select **programming language**
3. Select **framework**
4. Click **Generate Test Scripts** and the tool generate:

    - Multiple Script files
    - Reusable functions
    - Page-object models (if relevant)

All files are downloadable directly or as a ZIP.

4.4 Test Data Generation

Purpose

Creates synthetic or realistic datasets for testing.

Modes Supported

- Domain-based data
- Boundary data
- Negative data
- Combinatorial datasets
- Randomized or constraint-based data

Steps

- Choose input source from test cases or test scripts
- Select output format from
    - JSON
    - CSV
    - SQL
- Click **Generate Test Data** and tool returns:
    - JSON data objects to copy

    - SQL dataset to download

    - CSV dataset to download

5. Best Practices

For Test Cases:

✓ Provide complete requirements
✓ Choose a proper format to upload

For Test Scripts:

✓ Indicate environment details
✓ ~~Provide API specs if generating API tests~~

For Test Data:

✓ Mention correct input source (Choose Test Scripts if you have generated one)
✓ Specify format (CSV/JSON/SQL)

---

6. Troubleshooting

Issue	Possible Cause	Solution
No output generated	Missing requirement text	Ensure inputs are provided
Script errors	Unsupported test framework	Choose supported frameworks
Slow performance	Large document ingestion	Break into smaller sections
Wrong test cases	Ambiguous requirement text	Add clarifying notes

---

7. Why Choose QEAF

This framework enables:

- **Rapid onboarding of new clients and projects**
- **Consistent behaviour across environments**
- **Flexible deployment on-premises or in the cloud**

- **Long-term scalability and maintainability**

By leveraging configuration-driven customization, teams can meet diverse client needs while maintaining a single, robust framework foundation.

---

8. Conclusion

The QE Agentic AI tool provides an **end-to-end intelligent automation platform** for Quality Engineering teams. Its agent-based architecture, powered by LLMs and RAG, enables:

- Faster test case creation
- Reliable test data generation
- Automated script authoring
- Consistent requirement understanding

This dramatically accelerates QE activities and improves overall software quality.

---

9. FAQs

To be updated

---

10. Roadmap

Future: auto-healing automation, deeper integrations, multi-agent collaboration.
